Ders 1 Genişletilmiş Notu: Dersin İşleyişi, Algoritma Kavramı, Analiz Senaryoları ve Temel İşlem Sayma

Ders 1 Genişletilmiş Notu: Dersin İşleyişi, Algoritma Kavramı, Analiz Senaryoları ve Temel İşlem Sayma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı ve Kaynaklar
- 2. Dönem Boyunca Görülecek Konular
- 3. Algoritma Nedir?
- 4. Algoritmanın Karmaşıklığını Etkileyen Problem Parametreleri
- 5. Önemli Problem Türleri
- 6. Algoritmalarla İlgili Temel Sorular
- 7. Algoritma Tasarım Stratejileri ve Kaba Kuvvet
- 8. İyi Bir Algoritma: Doğruluk, Verimlilik ve Optimallik
- 9. Algoritmik Problemler Neden Önemli?
- 10. Algoritma Tasarımının Adımları
- 11. Algoritmayı Değerlendirme Ölçütleri
- 12. Karmaşıklık Senaryoları: En İyi, Ortalama ve En Kötü Durum
- 13. Senaryo Örnekleri: Sıralı Arama ve En Büyük Eleman
- 14. Deneysel Analiz
- 15. Kuramsal Analiz ve Sözde Kod
- 16. Temel İşlemler ve İşlem Sayma: arrayMax
- 17. Büyüme Derecesini Döngüler Belirler

1. Dersin Amacı ve Kaynaklar

Algoritma Analizi dersi iki grup hâlinde veriliyor. Grup 1'i Elif Hoca, Grup 2'yi (bu kayıt) dersin hocası veriyor. İki grup aynı konuları işliyor, ama anlatım stilleri farklı olduğu için küçük farklılıklar olabilir.

Hoca bu dönem derste yalnız slayt kullanmayacağını, bir tahta uygulamasında yazarak ilerleyeceğini söyledi. Öğrencilerden sınıftaki gibi defter tutmalarını istedi. Gerekçesi şu: Görerek ve duyarak öğreniyoruz, ama öğrendiğimizi kendimiz yapmadığımızda bilgi eksik kalıyor. Algoritma analizinde formüller ve analiz yöntemleri olduğu için bunları yazarak denemek gerekiyor.

> **Hocanın vurgusu:** Slaytlar ve kayıtlar paylaşılsa da dersin defterini tutun. Yazarak not alan bir adım öne geçer.

Dersin ana kitabı Levitin'in *Introduction to the Design and Analysis of Algorithms* kitabıdır. Ders ağırlıklı olarak bu kitaptan işlenecek. Destek kitapları Steven Skiena'nın ve Robert Sedgewick'in kitaplarıdır. Hoca bunları bilgisayar bilimlerinin çekirdek konusu olan algoritma konusunda "başucu kitapları" olarak tavsiye etti.

Bu ders, şimdiye kadar görülen algoritma derslerinden farklı olarak işin matematiğine ve teorisine iniyor. Hoca bunu "algoritmanın ameliyatını yapmak, içine açıp organlarına bakmak" diye benzetti. Dersin sonunda kazanılacaklar:

- Bir algoritmanın doğru çalışıp çalışmadığını göstermek.
- Bir algoritmanın verimli olup olmadığını anlamak.
- İki algoritmayı karmaşıklık açısından, kuramsal olarak ve farklı yollarla karşılaştırmak.

Kuramsal analizin önemi şudur: Bir algoritmanın kuramsal analizini yaptığınızda onu diğer algoritmalarla platformdan bağımsız (platform independent) olarak karşılaştırabilirsiniz. Karmaşıklığı kuramsal olarak ifade edebiliyorsak genelleştirmiş ve karşılaştırılabilir bir ortam kurmuş oluruz.

Hoca, derste daha önce görülmüş algoritmalarla da karşılaşılacağını söyledi. Amaç bu algoritmaları yeniden anlatmak değil, analizin nasıl yapılacağını bu algoritmalar üzerinden göstermektir. Ayrıca yeni algoritmalar da gösterilecek ve tasarım yaklaşımları arasındaki benzerlikler tartışılacak.

2. Dönem Boyunca Görülecek Konular

Hoca 14 haftalık planı "maraton" diye niteledi ve konuları sırayla anlattı:

- **İlk iki hafta:** Algoritma analizinin temelleri ve bir algoritmanın verimliliğini etkileyen parametreler.
- **Asimptotik analiz (asymptotic analysis):** Hocanın "çok önemli" dediği kavram.
- **İteratif ve özyinelemeli (recursive) algoritmaların analizi (3. ve 4. hafta):** Bir problem hem iteratif hem özyinelemeli çözülebilir. Bazen bir bakış açısı diğerine göre avantaj sağlar.
- **Böl ve yönet (divide and conquer):** En güzel örneği merge sort, ona eşlik eden quick sort. Bunun yanında azalt ve yönet (decrease and conquer) ve dönüştür ve yönet (transform and conquer) yaklaşımları.
- **Graf algoritmaları:** Birçok problem graflarla, düğüm ve kenarlarla ifade edilir; örneğin sosyal ağlar. Grafların analizi ve karmaşıklıkları konuşulacak.
- **Geri izleme (backtracking) ve dal-sınır (branch and bound):** Geri izleme denince 8 vezir (8 queens) problemi ve labirent problemi akla gelir. Bilgisayar girişten çıkışa nasıl gider, duvarla karşılaşınca ne yapar, çıkmaz sokağa girince nasıl davranır?
- **Dinamik programlama (dynamic programming):** Daha önce görülmemiş yeni bir yaklaşım. Bilinen bazı problemlerin nasıl çözülebileceği ve nerede kullanılabileceği.
- **Hash:** Hash'in ne olduğu, bir iki hafta.
- **Zaman-yer ödünleşimi (time-space trade-off):** Karmaşıklık denince zaman ve yer karmaşıklığı öne çıkar. Hangi durumda hangisini tercih etmek gerektiği konuşulacak.
- **P, NP, NP-complete, NP-hard:** En zor konulardan biri. Hoca, bu alanda hâlâ ispatı süren, çözülürse büyük ödüller konmuş sorular olduğunu söyledi.

  Ek bilgi (derste anlatılmadı): Hocanın sözünü ettiği ödüllü soru büyük olasılıkla P = NP sorusudur. Clay Matematik Enstitüsü bu soruyu büyük olasılıkla 2000 yılında Milenyum Problemleri arasına koymuş ve çözümüne bir milyon dolar ödül ayırmıştır.

3. Algoritma Nedir?

Hoca kitabın slaytlarına geçerek algoritmanın tanımını hatırlattı. Algoritma, bir problemi çözmek için **belirsiz olmayan (unambiguous)** adımlardan oluşan bir dizidir. Bir algoritma tasarlıyorsanız adımların çok net tarif edilmiş olması gerekir. Adımları okuyan kişi, tasarlayanın ne demek istediğini açıkça anlayabilmelidir. Hoca "unambiguous" kelimesinin çok önemli olduğunu vurguladı.

Algoritmadan ayrıca şunlar beklenir:

- Belirli bir süre içinde, yani sonlu sayıda adımda bitmesi.
- Geçerli her girdi için doğru çıktıyı üretmesi.

Hoca burada istenen sürede bitmeye "polinom zaman (polynomial time)" dediğini söyledi. Bazı problemlerin tam (exact) çözümü polinom zamanda elde edilir. Bazı problemlerde ise istenen sürede tam doğru sonucu üretme şansı olmayabilir. Bunlar için yaklaşık (approximate) sonuç üreten çözümler kullanılır. Yani her problemi her zaman, istenen zaman dilimi içinde optimal çözen bir algoritma tasarlayamayabiliriz.

> **Ders sonrası not:** Hoca, algoritmanın istenen sürede bitmesine polinom zaman dedi. Algoritma tanımındaki şart yalnızca sonlu sayıda adımda bitmektir. Polinom zaman ayrı bir kavramdır ve verimlilik sınıfını anlatır. Üstel zamanda çalışan bir yöntem de sonlu adımda bitiyorsa algoritmadır.

3.1 Aynı Problem, Farklı Algoritmalar: EBOB

Slayttaki örnek en büyük ortak böleni (EBOB, greatest common divisor, gcd) bulma problemiydi. 60 ile 24'ün en büyük ortak böleni 12'dir. Hoca bunu bulmak için iki farklı bakış açısı olduğunu söyledi.

Birinci yol: çarpanlara ayırma. İlkokul ve ortaokulda öğrendiğimiz yöntemdir. Sayıları çarpanlarına ayırır, ortak çarpanları alır ve çarparız (ara adım eklendi):

$$60 = 2^2 \cdot 3 \cdot 5, \qquad 24 = 2^3 \cdot 3, \qquad \gcd(60, 24) = 2^2 \cdot 3 = 12$$

İkinci yol: mod alarak ilerleme (Öklid algoritması). 60'ın 24'e göre modunu alıp böyle ilerleyerek sıfıra ulaşırız. Elimizde en son kalan sayı en büyük ortak bölendir (ara adımlar eklendi):

Adım	m	n	$m \bmod n$
1	60	24	12
2	24	12	0

Kalan 0 olduğunda son bölen 12'dir, yani $\gcd(60, 24) = 12$.

> **Ek bilgi (derste anlatılmadı):** Öklid algoritması $\gcd(m, n) = \gcd(n, m \bmod n)$ eşitliğine dayanır ve $n = 0$ olunca m'yi döndürür. Levitin'in kitabı bu algoritmayı büyük olasılıkla ilk bölümde, çarpanlara ayırma ve ardışık tamsayı deneme yöntemleriyle birlikte verir.

Hocanın çıkardığı sonuç: Bir problem için birden fazla algoritma olabilir. Bu yüzden önerilen algoritmaların ne kadar verimli olduğuna iyi bakmak gerekir. Yalnız verimlilik değil, algoritmanın anlaşılabilir ve genellenebilir olması da önemlidir.

Bir algoritmanın genel yapısı şöyledir: Bir problem vardır, bu problem için bir çözüm (algoritma) oluşturursunuz. Sonra bu çözüme girdiyi (input) verdiğinizde istediğiniz çıktıyı (output) alıp almadığınıza bakarsınız.

4. Algoritmanın Karmaşıklığını Etkileyen Problem Parametreleri

Hoca sınıfa algoritmanın karmaşıklığına nelerin etki ettiğini sordu.

İlk cevap **eleman sayısı** oldu. Hoca bunun en önemli parametre olduğunu söyledi. Algoritmanın karmaşıklığını özellikle eleman sayısı üzerinden ifade edeceğiz. Bin tane sayıyı sıralamakla bir milyon tane sayıyı sıralamak aynı şey değildir.

Soru: Bellek miktarı bilgisayara göre farklılık gösterebilir mi? **Cevap:** Olabilir, ama şu an bilgisayar tarafındaki parametreleri değil, problemi ele aldığımızda karmaşıklığa etki eden problem parametrelerini konuşuyoruz.

Hoca bir parametre daha olduğunu söyledi. "Elemanları ne sıklıkla incelediğimiz" cevabı tam doğru bulunmadı. Doğru cevap **elemanların ne olduğu ve hangi sırada olduğu** oldu. Yani elemanların içeriği (negatif mi, pozitif mi), veriliş sırası ve bu elemanları hangi sırayla aldığımız önemlidir.

Bu yüzden karmaşıklık incelenirken yalnız eleman sayısına değil, elemanların sırasına da bakılır. Karmaşıklık analizinde en iyi durum (best case), ortalama durum (average case) ve en kötü durum (worst case) senaryolarının konuşulmasının nedeni budur.

Hoca bunu sıralama algoritmalarıyla bağladı. Selection sort, insertion sort gibi birçok sıralama algoritması vardır. Elemanların nasıl dizildiğine bağlı olarak bunların birbirine göre artı ve

eksi yönleri vardır. Sıralıya yakın bir veri grubuyla çalışırken tercih edeceğimiz algoritma, çok karışık bir veri grubuyla çalışırken tercih edeceğimizden farklı olabilir.

5. Önemli Problem Türleri

Hoca slayttaki temel problem türlerinden bahsetti:

- **Sıralama (sorting):** En çok kullandığımız işlemlerden biri. Bir problemle uğraşmadan önce, onu daha rahat çözmek için verilen grubu genelde önce sıralarız. Bu yüzden sıralama algoritmaları bu derste kuramsal olarak analiz edilecek.
- **Arama (searching):** Çok yapılan işlerden biri.
- **Graflarda en kısa yol:** Çok popüler bir problem. Bir graf üzerinde çeşitli noktaları en kısa yoldan gezmek gibi.
- **Sırt çantası (knapsack) problemi** ve **Hanoi kuleleri (Towers of Hanoi):** Bu dönem kullanılacak ve üzerinde durulacak problemlerden.

6. Algoritmalarla İlgili Temel Sorular

Hoca algoritmalarla ilgili temel alanları sıraladı: Bir algoritmayı nasıl tasarlarız, nasıl ifade ederiz, doğruluğunu nasıl gösteririz, verimliliğini nasıl analiz ederiz ve optimal olup olmadığını nasıl anlarız?

6.1 Algoritmayı İfade Etme Yolları

Soru (hocadan): Bir algoritmayı kaç farklı şekilde ifade edebiliriz? Öğrencilerden gelen cevaplarla hoca tahtaya şu dört yolu yazdı:

- Akış diyagramı (flowchart).
- Sözlü anlatım (oral presentation).
- Sözde kod (pseudocode).
- Programlama dili (programming language).

Hoca temelde bu dört yaklaşımın benimsendiğini söyledi. Şimdiye kadar ağırlıklı olarak akış diyagramı kullanılmıştı. Bu derste sözde kod daha çok tercih edilecek. Sözde kod, doğal dile benzeyen ama onun kısaltılmış hâli olan, akış diyagramına göre daha kompakt ve popüler bir gösterimdir.

> **Hocanın vurgusu:** Sözde kod bu derste en çok karşılaşacağınız gösterim olacak. Sınavlarda ve quizlerde de karşınıza çıkabilir. Sözde kod yazmayı şimdiden öğrenmeye başlayın.

6.2 Doğruluk, Verimlilik, Optimallik

Algoritmayı ifade ettikten sonra **doğruluğunu** göstermek gerekir. Verilen her girdi için istenen çıktıyı üretiyorsa algoritmanın doğruluğundan söz edebiliriz. Bunun çeşitli yolları vardır. Doğruluk matematiksel olarak ispatlanabilir ya da durumlar gösterilerek ispatlanabilir. Bilgisayar bilimlerinde daha çok test senaryoları kullanılır: farklı girdiler için çıktılar üretilir ve bunlar sağlanıyorsa doğruluk gösterilmiş olur. Bazı problemler için bunun matematiksel karşılığı da vardır.

Doğruluktan sonra **verimliliğe** bakılır. Bu da kuramsal analiz ve deneysel (empirical) analiz olmak üzere iki şekilde yapılabilir. Son olarak da çözümün **optimal** olup olmadığı belirlenir.

7. Algoritma Tasarım Stratejileri ve Kaba Kuvvet

Ders boyunca farklı tasarım stratejileri görülecek. Slayttakiler:

- Kaba kuvvet (brute force).
- Böl ve yönet (divide and conquer), azalt ve yönet (decrease and conquer), dönüştür ve yönet (transform and conquer).
- Açgözlü yaklaşım (greedy approach).
- Dinamik programlama.

- Geri izleme (backtracking) ve dal-sınır (branch and bound).
- Zaman-yer ödünleşimi (space and time trade-offs).

7.1 Böl ve Yönet: Merge Sort

Böl ve yönet, bir problemi bütün olarak ele almak yerine alt problemlere bölerek çözmektir. Hoca örnek olarak merge sort'u anlattı: Sıralanacak diziyi ikiye böleriz, sonra her parçayı yine ikiye böleriz. Bu, en altta birer elemanlık parçalar kalana kadar sürer. Sonra iki parçayı küçük-büyük karşılaştırmasıyla birleştiririz: küçüğü başa, büyüğü sona alırız. Daha sonra ikilileri birleştiririz ve böyle devam ederiz.

7.2 Kaba Kuvvet

Soru (hocadan): Kaba kuvvet nedir? **Cevap:** Tek tek denemek, deneme yanılma. Hoca bunu tamamladı: Olası bütün seçenekleri deneyerek uygun seçeneği bulmaya çalışmaktır.

On elemanda kaba kuvvet sorun yaratmaz. Eleman sayısı artmaya başladığında, özellikle algoritmanın karmaşıklığı n^2, n^3 gibi ifade ediliyorsa, tek tek deneme yaklaşımı bir kâbusa dönüşür. Kabul edilebilir sürede çözüm olasılığı ortadan kalkar. Yine de kaba kuvvet, birçok problemde ilk akla gelen en basit yöntemdir.

Hocanın örneği şifre kırmaydı. Üç basamaklı bir kilidiniz varsa 000'dan başlayıp 999'a kadar bütün olasılıkları denersiniz. Zamanınız varsa bu sorun değildir, kilidi illa ki açarsınız. Ama basamak sayısı altıya çıktığında "işin rengi değişir" ve o kadar kısa sürede çözemeyebilirsiniz. Üç basamakta $10^3 = 1000$, altı basamakta $10^6 = 1\,000\,000$ olasılık vardır (ara adım eklendi).

8. İyi Bir Algoritma: Doğruluk, Verimlilik ve Optimallik

Bir algoritmanın iyi olup olmadığını değerlendirirken ölçütlerin başında doğru çalışması gelir. Hızlı çalışması ve zaman ya da yer açısından karmaşıklığının düşük olması ikinci önceliktir.

> **Hocanın vurgusu:** Birinci öncelik algoritmanın doğru çalışmasıdır. Zaman ve bellek verimliliği ondan sonra gelir.

Slayttaki üç ölçüt: doğruluk, zaman açısından verimlilik ve kullanılan bellek miktarı açısından verimlilik.

8.1 Optimallik ve Alt Sınır

İkinci bir soru, başka bir algoritmayla karşılaştırırken "daha iyisi var mı?" sorusudur. Bunu cevaplamak için çözümün sınırlarına, yani alt sınırına (lower bound) ve üst sınırına (upper bound) bakmak gerekir. Daha iyisi var mı diye baktığımız için burada önemli olan alt sınırdır. Bir problem için alt sınıra inen bir çözüm bulduysak ve bunu ispatlayabiliyorsak daha iyi bir algoritmanın peşinden koşmamıza gerek yoktur.

Hoca sınıfa şunu sordu: Karşılaştırmalı sıralama algoritmalarının çözümleri optimal midir, daha iyisi olup olmadığını biliyor musunuz? Örneğin selection sort optimal bir çözüm müdür?

Soru: Çözümler her problem için farklı optimallik özelliklerine sahip olmaz mı, çözüm kümesine göre farklılık göstermez mi? **Cevap:** Burada sıralama problemini karşılaştırmalı biçimde çözen algoritmalardan bahsediyoruz ve bu doğru bir cevap değil. Selection sort'un en kötü durumdaki karmaşıklığının n^2 olduğunu biliyoruz.

Öte yandan $n \log n$ büyüme derecesine sahip algoritmalar vardır: merge sort bunların başında gelir, heap sort da bunlardandır. Bunların kendi içinde başka farklı özellikleri de vardır. Karşılaştırmalı bir sıralama algoritmasının $n \log n$'den daha düşük bir büyüme derecesine sahip olamayacağını biliyoruz. Bu yüzden bu algoritmaların ötesine gidilebilecek bir çözüm görünmüyor. Hoca bunun analizlere başlandığında daha iyi anlaşılacağını söyledi.

> **Ders sonrası not:** "Karşılaştırmalı sıralama $n \log n$'den daha düşük olamaz" ifadesi en kötü durum için doğrudur: karşılaştırmalı sıralamanın en kötü durum alt sınırı $\Omega(n \log n)$'dir. En iyi durumda insertion sort gibi algoritmalar sıralı girdide $O(n)$

karşılaştırmayla bitebilir. Bu yüzden merge sort ve heap sort en kötü durumda optimaldir; selection sort her durumda $\Theta(n^2)$ karşılaştırma yaptığı için optimal değildir.

Hoca slayttaki algoritma özelliklerini de toparladı: belli bir adım sonunda tamamlanabilir olmak, belirsiz adım içermemek, belirli bir girdi alıp belirli bir çıktı üretmek ve doğru çalışmanın ötesinde etkili (effective) bir sonuca sahip olmak.

9. Algoritmik Problemler Neden Önemli?

Hoca iki sebep saydı:

- **Kuramsal sebep:** Algoritmik problemler bilgisayar bilimlerinin tam merkezindedir. Bilgisayar bilimlerinin hedefi, günlük hayattaki problemleri algoritmik bakış açısıyla çözmektir.
- **Pratik sebep:** Günlük hayattaki problemleri otomatikleştirerek bilgisayarların insan gücü gerektirmeden çözmesini sağlamak. Veri verildikten sonra ilgili programın, yazılımın ya da çerçevenin (framework) problemi insan müdahalesi olmadan çözmesi.

10. Algoritma Tasarımının Adımları

Aradan sonra hoca tahtaya algoritma tasarımının adımlarını yazdı:

1. Problemi anlamak.
2. Yaklaşımı seçmek: tam çözüm mü, yaklaşık çözüm mü? Buradan algoritma tasarım tekniğine geçilir.
3. Seçilen teknikten yola çıkarak algoritmayı tasarlamak. Problemi hangi yaklaşımla çözeceğimiz önemlidir.
4. Tasarlanan algoritmanın doğruluğunu göstermek. Girdiler verilip alınan çıktılar karşılaştırılabilir ya da doğruluk matematiksel olarak gösterilebilir.
5. Algoritmanın verimliliğini zaman ve yer açısından analiz etmek.
6. Son olarak kodlamak.

Hoca bu adımların hem en zor hem en kolay problemde işe yaradığını söyledi. Tam çözüm (exact solution) ile yaklaşık problem çözme (approximate problem solving) ayrımının kritik olduğunu, ama şimdilik ayrıntısına girmeyeceğini belirtti.

11. Algoritmayı Değerlendirme Ölçütleri

Hoca bir algoritmayı analiz ederken dikkat edilecek dört başlığı tahtaya yazdı:

1. Basitlik (simplicity).
2. Genellik (generality).
3. Zaman verimliliği (time efficiency).
4. Yer verimliliği (space efficiency).

Basitlik, algoritmanın kolay anlaşılabilir olmasıdır. Bir algoritmanın rahat anlaşılması ve rahat ifade edilmesi kodlama aşamasını kolaylaştırır. Bir analistin geliştirdiği algoritma basitse, onu kodlayanlar çok daha kısa sürede kodlar.

Genellik, bir problem için üretilen çözümün ne kadar çok probleme genelleştirilebildiğidir. Genelleştirilebildiği ölçüde çözüm o kadar değerlidir. Belirli bir sorunu çözmek yerine genel bir probleme cevap verebilen bir algoritma tasarlamak her zaman daha değerlidir. Algoritma incelenirken şu sorulmalıdır: Çözüm genelleştirilebilir mi, yoksa yalnız belli şartlarda mı uygulanabilir?

Soru (hocadan): Zaman verimliliği nedir? **Cevap:** Algoritmanın ne kadar hızlı çalışabildiği. Hoca bunu doğruladı: Zaman verimliliği algoritmanın zaman karmaşıklığını, yani çalışma süresini ifade eder. Girdi verildiğinde algoritmanın ne sürede cevap vereceğini ve başka bir algoritmayla kıyaslandığında daha kısa ya da daha uzun sürede sonuç verip vermediğini bu analizle anlarız.

Hocanın örneği: Selection sort'un karmaşıklığı n^2, merge sort'unki $n \log n$'dir. Bu yüzden merge sort'un zaman verimliliği selection sort'a göre daha yüksektir. Öte yandan merge sort ekstra bellek kullandığı için yer verimliliği daha düşüktür.

Soru (hocadan): Yer verimliliği nedir? **Cevap:** Bir algoritma ne kadar az bellek kullanıyorsa o kadar iyidir diyebilir miyiz? Hoca bunu şöyle tanımladı: Algoritmanın bellek kullanımı açısından nasıl bir performans sergilediğidir.

Hoca veri yapıları dersindeki graf gösterimlerini hatırlattı. Grafları göstermek için komşuluk matrisi (adjacency matrix) ve bağlı liste (linked list) kullanılıyordu. Kenar ve düğüm sayısına bağlı olarak biri diğerine göre avantaj sağlıyordu. Kenar sayısı çok az, düğüm sayısı fazlaysa komşuluk matrisi kullanmak yer verimliliği açısından bir problemdi; yer karmaşıklığı kötü bir çözüm üretmiş olurduk.

> **Ek bilgi (derste anlatılmadı):** V düğümlü, E kenarlı bir grafta komşuluk matrisi her zaman V^2 hücre kullanır. Komşuluk listesi ise $V + E$ ile orantılı yer kullanır. Bu yüzden kenarı az olan seyrek (sparse) graflarda liste gösterimi daha az bellek ister.

12. Karmaşıklık Senaryoları: En İyi, Ortalama ve En Kötü Durum

Soru (hocadan): Bir algoritmanın karmaşıklığında kaç senaryo incelemeliyiz? **Cevap:** En kötü durum, en iyi durum ve ortalama durum. Hoca üç senaryoyu sıraladı: en iyi durum (best case), ortalama durum (average case), en kötü durum (worst case). Bir algoritmanın karmaşıklığından bahsediyorsak hangi durum için bahsettiğimizin adını koymamız gerekir.

Soru (hocadan): Bunlardan en kolay hesaplanabilen hangisi, en iyi durum işimize yarar mı? **Cevap (öğrenci):** Duruma göre değişebilir. Hoca genelde en iyi durumu kullanmadığımızı söyledi. En iyi durum, verilen girdinin türüne ve sırasına bağlıdır, arada bir denk gelebilir. Bir algoritmayı en iyi durum üzerinden karşılaştırırsak, en kötü durum geldiğinde çok daha kötü bir karmaşıklık ortaya çıkabilir ve biz bunu gözden kaçırabiliriz. Bu yüzden en iyi durum ilk odak noktamız değildir.

Soru (hocadan): Ortalama durumu mu, en kötü durumu mu hesaplamak daha zordur? **Cevap:** Ortalama durum her zaman daha zordur. Ortalama durumda her türlü olasılığı göz önünde bulundurup, oluşabilecek bütün olasılıkların ortalama senaryosunu bularak karmaşıklığı hesaplamanız gerekir. Bu her problemde kolay sağlanamaz. Bu yüzden ortalama durum genelde pek hesaplanmaz. Hoca ilerleyen zamanlarda bununla ilgili bir örnek yapılacağını söyledi.

Sonuçta en kritik senaryo en kötü durumdur: olabilecek en kötü senaryo.

Soru (hocadan): Bir sıralama algoritması için olabilecek en kötü senaryo nedir? **Cevap:** Dizinin tersten sıralı olması. En iyi senaryo, dizinin sıralamak istediğimiz yönde zaten sıralı verilmesidir. Ortalama durum ise karışık verilme senaryosudur. Bazı ortalama durumlarda ilk iki eleman sıralı olur, bazılarında yarısı sıralı olur, bazılarında hiçbiri sıralı olmaz. Ortalama durumu hesaplarken bütün bu durumların olma olasılığını düşünerek analiz yapmak gerekir.

> **Hocanın vurgusu:** Karmaşıklıktan bahsederken hangi senaryo için ifade ettiğinizi mutlaka söylemelisiniz.

12.1 Neden En Kötü Durum?

Hoca slayttaki gerekçeleri tekrarladı: En iyi durum genelde işe yaramaz, ortalama durum kolay belirlenemez. Bu derste odaklanılacak senaryo en kötü durumdur, çünkü:

- Analiz etmesi kolaydır.
- Birçok uygulamada donanım ve yazılım zaten en kötü durum düşünülerek tasarlanır. Slayt bunun oyunlar, finans ve robotik gibi uygulamalar için çok önemli olduğunu söylüyordu.

Hoca bunu üniversitenin öğrenci bilgi sistemiyle örnekledi. **Soru (hocadan):** Ders kayıt sisteminde en kötü durum nedir? **Cevap:** Herkesin aynı anda girmesi. Hoca daha somut bir cevap, en kötü durumdaki eleman sayısını istedi. Öğrenci sayısının yaklaşık 35 bin olduğu söylendi. **Soru:** En kötü durum 35 bin kişi diyebilir miyiz? **Cevap:** Hayır. Kayıt kurallarında yükü hafifletmek için her sınıf ayrı bir günde ders seçiyor. Bu yüzden sistemi tasarlarken en kötü durum olarak alınacak örnek, sınıflar içindeki en kalabalık sınıftır. Dördüncü sınıflar en kalabalıksa tasarım, dizayn ve ölçüm dördüncü sınıftaki kişi sayısı esas alınarak yapılmalıdır.

Hoca bunun alt seviye algoritmik bakış açısıyla birebir örtüşmeyebileceğini, ama gerçek hayattaki en kötü durum senaryolarının da böyle düşünüldüğünü söyledi. Bir başka örnek: ÖSYM

sonuçları açıklanırken en kötü durum, 2 milyon kişinin aynı anda sonuçlarına erişmek istemesidir. Uygulamanın performans testi bu tür analizlerle belirlenir.

13. Senaryo Örnekleri: Sıralı Arama ve En Büyük Eleman

13.1 Sıralı Arama

Soru (hocadan): n elemanlı bir dizide sıralı arama (sequential search, linear search) yapıldığında karmaşıklık nedir? İlk gelen cevap "n" oldu ve hoca bunu "yanlış cevap" diye değerlendirdi, çünkü senaryo söylenmemişti. "n^2" cevabına hoca selection sort'u değil sıralı aramayı sorduğunu söyledi. Sonunda doğru cevap geldi: En iyi durum $O(1)$, en kötü durum $O(n)$.

Hoca tahtaya iki dizi yazdı:

- Sırasız (unsorted): `1 5 3 2 4 8`
- Sıralı (sorted): `1 2 3 4 5 8`

Sorulması gereken ilk soru, elemanların sıralı olup olmadığıdır. Bir soruda "verilen bir dizi" denip sıralı olduğu söylenmemişse dizi sırasız kabul edilir. Eleman sayısı her iki dizide de n olarak alınır.

Sırasız dizi. En kötü durumda her elemana bakmak gerekir, karmaşıklık $O(n)$'dir. Hoca bunu Big-O cinsinden, yani üst sınır (upper bound) olarak söylediğini belirtti. En iyi durumda aradığımız eleman ilk sırada olabilir. Örneğin 1'i arıyorsak ilk adımda buluruz, karmaşıklık $O(1)$'dir.

Sıralı dizi. Soru (hocadan): Dizi sıralı olsaydı ve yine sıralı arama yapsaydık en kötü durum ne olurdu? **Cevap (öğrenci):** $O(\log n)$, çünkü ikili arama (binary search) kullanabiliriz. Hoca ikili arama demediğini, sorunun sıralı arama için olduğunu söyledi. Arama sırayla yapıldığı için en kötü durum yine $O(n)$'dir. İkili arama olsaydı cevap farklı olurdu. En iyi durum yine $O(1)$'dir.

> **Hocanın vurgusu:** Burada kullandığımız gösterimin Big-O gösterimi olduğunu notlarınıza yazın.

13.2 Dizide En Büyük Elemanı Bulma

Hoca tahtayı temizleyip ikinci problemi yazdı: dizide en büyük elemanı bulma (finding maximum within array). Eleman sayısı yine n, algoritma da bildiğimiz standart maksimum bulma algoritmasıdır.

Soru (öğrenci): Dizi sıralı mı? Hoca bunun doğru soru olduğunu söyledi ve iki durumu ayrı yazdı.

Sırasız dizi. "Sırasız için $O(n)$" cevabını hoca "yanlış cevap" diye niteledi, çünkü senaryo belirtilmemişti. Doğrusu: **bütün senaryolarda** $O(n)$. En kötü durumda da en iyi durumda da fark yoktur. Bunun sıralı aramadan farkı şudur: Orada "dizide bu eleman var mı?" diye soruyorduk. Burada "en büyük eleman hangisi?" diye soruyoruz. Sırasız bir dizide en büyük elemanı bulmanın tek yolu bütün elemanlara bakmaktır. Bir elemana bakıp "dizinin en büyük elemanını bulduk" diyemeyiz. Bu yüzden en iyi durumda da bütün elemanlara bakmak zorundayız.

Sıralı dizi. "$O(1)$" cevabına hoca "ne için?" diye sordu. "Bütün durumlarda $O(1)$" cevabı doğruydu. Dizinin sıralı olduğunu bildiğimiz için sondaki elemanı en büyük eleman olarak alırız. Bunun için dizinin hangi yöne sıralı olduğunu, küçükten büyüğe mi büyükten küçüğe mi, bilmemiz gerekir. Bunu biliyorsak birinci ya da n'inci elemanı alarak tek hamlede maksimuma ulaşırız. En kötü durumda da durum değişmez.

> **Hocanın vurgusu:** "Bütün durumlarda" cümlesini kurmazsanız verdiğiniz cevap yanlış sayılır. Bu kadar basit örnekler üzerinden gitmenin sebebi bu kavramları oturtmanızdır.

Tahtadaki sonuçların özeti:

Problem	Dizi	En iyi durum	En kötü durum
Sıralı arama	Sırasız	$O(1)$	$O(n)$
Sıralı arama	Sıralı	$O(1)$	$O(n)$
En büyük eleman	Sırasız	$O(n)$	$O(n)$
En büyük eleman	Sıralı	$O(1)$	$O(1)$

14. Deneysel Analiz

Hoca slaytlarla şimdiye kadar konuşulanları toparladı: Analizde dikkat edilecek başlıklar, kaç farklı senaryo olduğu, zaman ve yer verimliliği. Sonra algoritmaları karşılaştırmanın iki yolunu söyledi: kuramsal analiz ve pratik (deneysel) analiz. Kuramsal analizde temel işlemlerden (basic operations) bahsetmek, bunları saymak ve sonunda asimptotik analizle algoritmanın büyüme derecesine karar vermek gerekir.

İki algoritmayı karşılaştırırken şunlara bakılır:

- Basit mi? Kolay uygulanabilir (implement), kolay anlaşılır, kolay değiştirilebilir mi? Bu, basitliğe girer.
- Ne kadar sürede sonlanıyor? Bu, zaman verimliliğine girer.
- Ne kadar bellek ihtiyacı var? Bu, yer verimliliğine girer.

Ders boyunca ağırlıklı olarak zaman verimliliği ve büyüme dereceleri tartışılacak. Slayttaki tanımlar:

- **Zaman karmaşıklığı (time complexity):** Bir algoritmanın tamamlanması için gereken süre.
- **Yer karmaşıklığı (space complexity):** Bir algoritmanın çalışması için gereken bellek miktarı.

Slaytta bir sıralama örneği vardı: Girdi `5 3 1 2`, çıktı `1 2 3 5`. Hedef, algoritmanın büyüme derecesini bir fonksiyon cinsinden ifade etmektir.

Soru (hocadan): Süreyi bir fonksiyonla ifade etme bilgimiz olmasaydı, iki algoritmanın zaman karmaşıklığını nasıl karşılaştırırdık? **Cevap:** Kodun içine ekleyerek çalışma sürelerini kıyaslayabiliriz. Hoca bunun yapılabilecek en güzel çözümlerden biri olduğunu söyledi: İki algoritmayı arka arkaya çalıştırıp sürelerini not etmek. Programın başladığı yerde saat bilgisi alınır, bittiği anda bir saat bilgisi daha alınır ve aradaki farka bakılır.

Soru (hocadan): Böyle bir yaklaşımda nelere dikkat etmek gerekir? Öğrencilerden gelen cevaplar ve hocanın değerlendirmesi:

- **Girdiler aynı olmalı.** Hoca bunu birinci koşul olarak kabul etti.
- **Veriyi temizlemek, kıyaslamaya hazır etmek.** Hoca bunun çok gerekli olmadığını söyledi. Aynı veriyi veriyorsak özel bir şey yapmaya gerek yok. Çok büyük verilerle çalışıyorsak veri temizliği her zaman yapılabilir.
- **Verinin zaten sıralı olup olmadığını, yani senaryoyu bilmek.** Aynı veriyi verdiğimiz için zaten aynı senaryoyu karşılıyoruz.
- **İki algoritmanın da doğru sonuç verdiğini kontrol etmek.** Hoca buna itiraz etmedi, ama şu an zaman karmaşıklığı açısından hangisinin avantajlı olduğunu bulmaya çalıştığımızı hatırlattı.
- **İki ölçümde bilgisayarın eş güçlü olması.** Hoca bunun çok kritik olduğunu söyledi: İki algoritma donanım (hardware) ve yazılım (software) olarak aynı ortamda çalıştırılmalıdır.

Bunun ötesinde algoritmaların kodlanması gerekir. Hoca bunun bir yük olduğunu, pratik ölçüm için iki algoritmanın da bir programlama diliyle aynı ortamda kodlanması gerektiğini söyledi. Slayt ayrıca girdiler aynı boyutta olsa bile içerikleri farklıysa çalışma sürelerinin farklı olabileceğini belirtiyordu.

Deneysel çalışmanın adımları:

1. Karşılaştırılacak algoritmaların programını yazmak.
2. Programları farklı girdi boyutları için çalıştırmak.
3. Uygun fonksiyonlarla süreleri ölçmek ve kıyaslamak.

4. Sonuçları grafiğe dökmek (plot). Slayttaki grafikte aşağı yukarı ikinci dereceden (quadratic) bir yükseliş görülüyordu.

14.1 Deneysel Analizin Sınırları

- Algoritmaların mutlaka kodlanması gerekir.
- Bazı girdi türleri ve dizilimler denenmemiş olabilir. Her olasılığı deneme şansımız yoktur, gözden kaçan olasılıklar olur. Bu yüzden sonuç yalnız denenen girdiler için geçerlidir.
- İki algoritmayı karşılaştırıyorsanız ölçümleri aynı donanım ve yazılım ortamında yapmanız gerekir.

15. Kuramsal Analiz ve Sözde Kod

Kuramsal analizde kod yerine sözde kod açıklamaları kullanılır. Girdi boyutu genelde küçük n ile gösterilir; zaman zaman büyük N da kullanılır. İfadeler n cinsinden değerlendirilir. Kuramsal analizin avantajları:

- Bütün olası girdiler işin içine katılmış olur.
- Dizilimleri ayrıca uygulayıp ölçmek gerekmez.
- Donanım ve yazılımdan bağımsızdır. Bu sayede iki algoritmanın çalışma süresini tamamen nesnel biçimde karşılaştırabiliriz.

15.1 Sözde Kod Kuralları

Slayta göre sözde kod, bir algoritmanın üst düzey açıklamasıdır (high-level description of an algorithm). Hoca kuralların çok basit olduğunu söyledi:

- Atama için eşittir yerine genelde ok kullanılır: $\leftarrow$.
- Eşitlik karşılaştırması için tek bir $=$ kullanılır.
- Dizi gösterimi bildiğimiz gibidir: `A[i]`.
- `for` ifadesi `to` ve `do` ile kullanılır.
- `if ... then ... else` aynı şekilde kullanılır.
- Kullanılabilecek diğer ifadeler: `while ... do`, `repeat ... until`.
- `return` bir fonksiyondan değer döndürmenin yoludur.
- Parametreler, girdiler (input) ve çıktılar (output) bildirilebilir.
- Bloklar parantezle değil girintiyle (indentation) gösterilir.

Slayttaki örnek `arrayMax` algoritmasıydı. Girdi olarak n elemanlı `A` dizisi verilir. Çıktı olarak dizinin en büyük elemanı döndürülür.

Hoca, akış diyagramında "böyle kullanamazsınız" denen bazı matematiksel gösterimlerin sözde kodda kullanılabildiğini söyledi. Sözde kod, algoritmik tasarımı çok daha hızlı ifade etmeyi sağlar. İş hayatında da sık karşılaşılır. Akış diyagramı uzun vadede katkı sağlasa da gösterim açısından daha modern yöntem sözde koddur.

16. Temel İşlemler ve İşlem Sayma: arrayMax

Kuramsal analizde ilgilendiğimiz işlemler, temel (primitive, basic) işlemlerdir.

Soru (hocadan): Temel işlemler nelerdir? Öğrencilerden gelen cevaplarla hoca şunları saydı: dört işlem (aritmetik işlemler), karşılaştırma, döngüler ve dönüş (return). Hoca döngülerin aslında karşılaştırma ve atama işlemlerini içerdiğini, atamanın da temel bir işlem olduğunu söyledi. Ardından dönüşün tam karşılığını sordu. **Cevap:** Fonksiyon çağırmak.

Slayttaki temel işlem listesi:

- Bir ifadeyi değerlendirmek (evaluating an expression): aritmetik işlemler ve karşılaştırma (relational) işlemleri.
- Bir değişkene değer atamak (assigning a value).
- Diziye erişmek (indexing into an array). Bu bir bellek erişimi olduğu için her erişim bir işlem sayılır.
- Bir metodu çağırmak (calling a method).
- Bir metottan dönmek (returning from a method).

> **Hocanın vurgusu:** Bir algoritmayı kuramsal olarak analiz etmek için temel işlemlerin sayılmasına ihtiyacımız var.

16.1 arrayMax'ta İşlem Sayma

Hoca slayttaki sayımı satır satır anlattı. İlk satırda `currentMax`'a bir atama vardır, bu bir işlemdir. `A`'ya bir kez erişim de bir işlemdir; satır toplam iki işlem olarak hesaplanır. Döngü içindeki karşılaştırmalar, erişimler ve diğer işler hesaplanarak her satırda kaç işlem yapıldığına karar verilir. Toplamda maksimum bulma algoritmasında $7n - 1$ işlem yapıldığı bulunur.

```
Algorithm arrayMax(A, n)                # işlem sayısı
  currentMax <- A[0]                    2
  for i <- 1 to n - 1 do                2 + n
    if A[i] > currentMax then           2(n - 1)
      currentMax <- A[i]                2(n - 1)
    { increment counter i }             2(n - 1)
  return currentMax                     1
                                Toplam: 7n - 1
```

Toplamın hesabı (ara adım eklendi):

$$T(n) = 2 + (2 + n) + 2(n-1) + 2(n-1) + 2(n-1) + 1 = 5 + n + 6n - 6 = 7n - 1$$

Bu sayım, karşılaştırmanın her seferinde doğru çıktığı ve `currentMax`'ın her adımda güncellendiği en kötü durum içindir. Girdi boyutu olarak n kullanılır ve fonksiyon bu n cinsinden ifade edilir.

> **Ders sonrası not:** $7n - 1$ sayısı python ile işlem sayan bir simülasyonla doğrulandı (artan dizide $n = 10$ için 69). En iyi durumda, ilk eleman en büyükse, atama satırı hiç çalışmaz ve toplam $7n - 1 - 2(n-1) = 5n + 1$ olur ($n = 10$ için 51).

16.2 Gerçek Çalışma Süresinin Sınırları

Slayttaki açıklamaya göre en kötü durumda $T(n)$, temel işlemler arasındaki en hızlı ve en yavaş çalışan işlemin belirlediği sınırlar arasındadır; uç noktalarda da olabilir. Hangi işlemin en hızlı, hangisinin en yavaş çalıştığını bilmeyiz, ama bu sınırlar arasında olduğunu biliriz. En hızlı temel işlemin süresi a, en yavaşınınki b ise (ara adım eklendi):

$$a(7n - 1) \leq T(n) \leq b(7n - 1)$$

Her iki durumda da $T(n)$ doğrusal (linear) bir fonksiyondur.

17. Büyüme Derecesini Döngüler Belirler

Soru (hocadan): Bu algoritmanın doğrusal büyüdüğünü sözde koda bakarak nereden anlarız? **Cevap (öğrenci):** En büyük kuvvetin bir olmasından. Hoca bu ifadeyi tercih etmediğini söyledi ve cevabın "tam karşınızda durduğunu" belirtti. Doğru cevap: **Tek bir `for` döngüsü olduğu için.**

İç içe döngü yoktur, döngü n kez döner. İç içe döngü olsaydı içerideki işlemler n^2 kez çalışacağı için büyüme otomatik olarak n^2'ye doğru giderdi. Diğer temel işlemler zaten bir kez gerçekleşir. Döngü içindekiler ise, her algoritmada böyle olmasa da genelde eleman sayısıyla doğru orantılı olarak çalışır.

> **Hocanın vurgusu:** Bir algoritmanın büyüme derecesini bulmaya çalışırken döngüler en belirleyici özelliktir. Burası önemli.

Örnek Sorular

1. (derste sözlü, 01:14:48) Bir algoritmanın karmaşıklığına etki eden problem parametreleri nelerdir?

> **Çözüm (derste verildi):** Eleman sayısı (n) ile elemanların içeriği ve veriliş sırası. Bilgisayar tarafındaki bellek gibi parametreler bu soruya girmez. Sıranın etkisi yüzünden en iyi, ortalama ve en kötü durum ayrı incelenir.

2. (derste sözlü, 01:21:11) Bir algoritmayı kaç farklı şekilde ifade edebiliriz?

> **Çözüm (derste verildi):** Dört yolla: akış diyagramı, sözlü anlatım, sözde kod ve programlama dili.

3. (derste sözlü, 01:27:20) Kaba kuvvet (brute force) stratejisi nedir? Ne zaman sorun yaratır?

> **Çözüm (derste verildi):** Olası bütün seçenekleri deneyerek uygun olanı bulmaktır. Eleman sayısı arttıkça deneme sayısı hızla büyür ve kabul edilebilir sürede çözüm imkânsızlaşır. Örnek: 3 basamaklı kilitte 000-999 arası denenir; 6 basamakta iş çok uzar.

4. (derste sözlü, 01:31:18) Karşılaştırmalı sıralama algoritmaları optimal midir? Selection sort optimal bir çözüm müdür?

> **Çözüm (derste verildi):** Selection sort optimal değildir, en kötü durumda n^2'dir. Karşılaştırmalı bir sıralama $n \log n$'den daha iyi olamaz (en kötü durum alt sınırı). Merge sort ve heap sort $n \log n$ olduğu için en kötü durumda optimaldir.

5. (derste sözlü, 01:47:23 ve 01:48:45) Zaman verimliliği ve yer verimliliği nedir? Merge sort ile selection sort'u bu iki ölçüte göre karşılaştırın.

> **Çözüm (derste verildi):** Zaman verimliliği algoritmanın çalışma süresini, yer verimliliği bellek kullanımını anlatır. Merge sort ($n \log n$) zaman açısından selection sort'tan (n^2) daha verimlidir. Ekstra bellek kullandığı için yer verimliliği daha düşüktür.

6. (derste sözlü, 01:50:00 - 01:52:00) Karmaşıklık analizinde hangi senaryolar incelenir? En iyi durum işimize yarar mı? Ortalama durum mu, en kötü durum mu daha zor hesaplanır?

> **Çözüm (derste verildi):** En iyi, ortalama ve en kötü durum. En iyi durum yalnız belirli girdilerde denk geldiği için tek başına yanıltıcıdır. Ortalama durum, bütün olasılıkların hesaba katılmasını gerektirdiği için her zaman daha zordur. Derste esas alınan en kötü durumdur.

7. (derste sözlü, 01:52:57) Bir sıralama algoritması için en kötü ve en iyi senaryo nedir?

> **Çözüm (derste verildi):** En kötü senaryo dizinin tersten sıralı olması, en iyi senaryo istenen yönde zaten sıralı olmasıdır. Ortalama durum karışık sırayla verilmesidir.

8. (derste sözlü, 01:54:05) n elemanlı bir dizide sıralı aramanın (linear search) karmaşıklığı nedir? Dizi sıralı olsaydı en kötü durum ne olurdu?

> **Çözüm (derste verildi):** Senaryo belirtilmeden verilen "$O(n)$" cevabı yanlıştır. Sırasız dizide en iyi durum $O(1)$, en kötü durum $O(n)$. Dizi sıralı olsa da sıralı aramada en kötü durum $O(n)$, en iyi durum $O(1)$'dir. $O(\log n)$ ikili aramanın cevabıdır.

9. (derste sözlü, 01:58:41) Bir dizide en büyük elemanı bulmanın karmaşıklığı nedir? Sırasız ve sıralı diziyi ayrı değerlendirin.

> **Çözüm (derste verildi):** Sırasız dizide bütün durumlarda $O(n)$, çünkü bütün elemanlara bakmak gerekir. Sıralı dizide, sıralama yönü biliniyorsa, bütün durumlarda $O(1)$: ilk ya da son eleman alınır.

10. (derste sözlü, 02:06:16) İki algoritmanın çalışma süresini deneysel olarak karşılaştırırken nelere dikkat etmek gerekir?

Çözüm (derste verildi): Girdiler aynı olmalı. İki algoritma aynı donanım ve yazılım ortamında çalıştırılmalı. Algoritmalar önceden kodlanmalı. Bütün girdiler denenemeyeceği için sonuç yalnız denenen girdiler için geçerlidir.

11. (derste sözlü, 02:09:34) Üniversitenin öğrenci bilgi sisteminde ders kaydı için en kötü durumdaki kullanıcı sayısı nedir? Toplam öğrenci sayısı (yaklaşık 35 bin) denebilir mi?

Çözüm (derste verildi): Denemez. Her sınıf ders kaydını ayrı bir günde yapıyor. En kötü durum, en kalabalık sınıftaki öğrenci sayısıdır; sistem buna göre tasarlanır.

12. (derste sözlü, 02:19:00) Sözde kodda temel (primitive) işlemler nelerdir?

Çözüm (derste verildi): İfade değerlendirme (aritmetik ve karşılaştırma işlemleri), atama, diziye erişim, metot çağırma ve metottan dönme.

13. (derste sözlü, 02:24:08) `arrayMax` algoritmasının doğrusal büyüdüğünü sözde koda bakarak nereden anlarız?

Çözüm (derste verildi): Algoritmada tek bir döngü vardır ve n kez döner; iç içe döngü yoktur. Diğer işlemler bir kez çalışır. İç içe iki döngü olsaydı büyüme n^2 olurdu.

Dersin Toparlaması

Bu derste algoritmanın tanımıyla başlandı: belirsiz olmayan, sonlu adımda biten ve geçerli her girdi için doğru çıktı üreten adımlar dizisi. Aynı problemin EBOB örneğinde olduğu gibi farklı algoritmalarla çözülebileceği görüldü. Karmaşıklığı etkileyen problem parametrelerinin eleman sayısı ile elemanların içeriği ve sırası olduğu, bu yüzden en iyi, ortalama ve en kötü durumun ayrı incelendiği anlatıldı. Bir algoritma doğruluk, basitlik, genellik, zaman ve yer verimliliği ile optimallik açısından değerlendirilir. Derste esas alınacak senaryo en kötü durumdur ve karmaşıklık söylenirken senaryo mutlaka belirtilmelidir.

Algoritmaları karşılaştırmanın iki yolu ele alındı. Deneysel analiz kodlama ve aynı ortam gerektirir ve bütün girdileri kapsayamaz. Kuramsal analiz ise sözde kod üzerinden temel işlemleri sayarak donanımdan bağımsız bir $T(n)$ fonksiyonu verir. `arrayMax` örneğinde bu fonksiyon $7n - 1$ çıktı ve büyümenin doğrusal olduğu, tek döngüden kaynaklandığı görüldü. Hoca, gelecek hafta slaytları tamamladıktan sonra büyüme derecelerine ve asimptotik gösterimlere (Big-O, Big-Theta, Big-Omega) geçileceğini söyledi. $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$ gibi yaklaşık 7-8 temel büyüme derecesi ve bunların birbirine göre hangisinin daha iyi olduğu konuşulacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.